2026 春

程序设计实习
第一讲

类和对象1

杨　帅
williamyang@pku.edu.cn

目　录

- **从C到C++**
  - 引用的概念和应用
  - const关键字的用法
  - 动态内存分配
  - 函数重载、函数缺省参数
- **类和对象**

目　录

- 从C到C++
  - 引用的概念和应用
  - const关键字的用法
  - 动态内存分配
  - 函数重载、函数缺省参数
- 类和对象

引用的概念

- 下面的写法定义了一个引用，并将其初始化为引用某个变量

类型名 & 引用名 = 某变量名;

引用的概念

- 下面的写法定义了一个引用，

  并将其初始化为引用某个变量

类型名 & 引用名 = 某变量名;

```
int n = 4;
int & r = n;  // r引用了 n, r的类型是
```

引用的概念

- 下面的写法定义了一个引用，

  并将其初始化为引用某个变量

类型名 & 引用名 = 某变量名;

```
int n = 4;
int & r = n;  // r引用了 n，r的类型是
```

某个变量的引用，等价于这个变量，相当于该变量的一个别名

引用的概念


```
int n = 4;
int & r = n;
r = 4;
cout << r;
cout << n;
n = 5;
cout << r;
```

引用的概念


```
int n = 4;
int & r = n;
r = 4;
cout << r;  //输出 4
cout << n;
n = 5;
cout << r;
```

引用的概念


```
int n = 4;
int & r = n;
r = 4;
cout << r;  //输出 4
cout << n;  //输出 4
n = 5;
cout << r;
```

引用的概念


```
int n = 4;
int & r = n;
r = 4;
cout << r;  //输出 4
cout << n;  //输出 4
n = 5;
cout << r;  //输出 5
```

引用的概念

- 定义引用时一定要将其**初始化成引用某个变量**
- 初始化后，它就一直引用该变量，不会再引用别的变量了
- 引用只能引用**变量**，不能引用常量和表达式。

引用的概念


```
double a = 4, b = 5;
double & r1 = a;
double & r2 = r1;
r2 = 10;
cout << a << endl;
r1 = b;
cout << a << endl;
```

引用的概念


```
double a = 4, b = 5;
double & r1 = a;
double & r2 = r1;   // r2也引用 a
r2 = 10;
cout << a << endl;
r1 = b;
cout << a << endl;
```

引用的概念


```
double a = 4, b = 5;
double & r1 = a;
double & r2 = r1;   // r2也引用 a
r2 = 10;
cout << a << endl;  // 输出 10
r1 = b;
cout << a << endl;
```

引用的概念


```
double a = 4, b = 5;
double & r1 = a;
double & r2 = r1;    // r2也引用 a
r2 = 10;
cout << a << endl;   // 输出 10
r1 = b;              // r1并没有引用b
cout << a << endl;
```

引用的概念


```
double a = 4, b = 5;
double & r1 = a;
double & r2 = r1;    // r2也引用 a
r2 = 10;
cout << a << endl;   // 输出 10
r1 = b;              // r1并没有引用b
cout << a << endl;   // 输出 5
```

引用应用的简单示例

- C语言中，如何编写交换两个整型变量值的函数?

引用应用的简单示例

- C语言中，如何编写交换两个整型变量值的函数?

```
void swap( int * a, int * b)
{
    int tmp;
    tmp = * a; * a = * b; * b = tmp;
}
int n1, n2;
swap(& n1, & n2); // n1,n2的值被交换
```

引用应用的简单示例

- 有了C++的引用

```
void swap( int & a, int & b)
{
    int tmp;
    tmp = a; a = b; b = tmp;
}
int n1, n2;
swap(n1, n2); // n1,n2的值被交换
```

引用作为函数的返回值

- 函数的返回值可以是引用，例：

```
#include <iostream>
using namespace std;
int n = 4;
int & SetValue()  {    return n;  }
//返回对n的引用
int main() {
    SetValue() = 40; //对返回值进行赋值 → 对n赋值
    cout << n;
    return 0;
} //程序输出结果是40
```

常引用

- 定义引用时，前面加const关键字，即为“常引用”

```
int n;
const int & r = n;
```

常引用

- 定义引用时，前面加const关键字，即为“常引用”

```
int n;
const int & r = n;
```

- r 的类型是 **const int &**
- 不能通过常引用去修改其引用的内容：

```
int n = 100;
const int & r = n;
r = 200;    //编译错
n = 300;   // 没问题
```

常引用

- 请注意，const T & 和 T & 是不同的类型
- T & 类型的引用或T类型的变量可以用来初始化const T &类型的引用
- const T类型的常变量和const T &类型的引用则不能用来初始化T &类型的引用，除非进行强制类型转换

测一测

- 下面程序片段哪个没错?

  A. int n = 4; int & r = n * 5;

  B. int n = 6; const int & r = n; r = 7;

  C. int n = 8; const int & r1 = n; int & r2 = r1;

  D. int n = 8; int & r1 = n; const int r2 = r1;

测一测

- 下面程序片段哪个没错?

A. int n = 4; int & r = n * 5;

B. int n = 6; const int & r = n; r = 7;

C. int n = 8; const int & r1 = n; int & r2 = r1;

D. int n = 8; int & r1 = n; const int r2 = r1;

测一测

- 下面程序的输出结果是什么：

```
int a = 1, b = 2;
int & r = a;
r = b;
r = 7;
cout << a << endl;
return 0;
```

A. 1
B. 2
C. 7
D. 都不是

测一测

- 下面程序的输出结果是什么：

```
int a = 1, b = 2;
int & r = a;
r = b;
r = 7;
cout << a << endl;
return 0;
```

A. 1

B. 2

C. 7

D. 都不是

目　录

- 从C到C++
  - 引用的概念和应用
  - const关键字的用法
  - 动态内存分配
  - 函数重载、函数缺省参数
- 类和对象

定义常量


```
const int MAX_VAL = 23;
const string SCHOOL_NAME = "Peking University";
```

定义常量指针

- 不可通过常量指针修改其指向的内容

```
int n, m;
const int * p = & n;
* p = 5;
n = 4;
p = &m;
```

定义常量指针

- 不可通过常量指针修改其指向的内容

```
int n, m;
const int * p = & n;
* p = 5;  //编译出错
n = 4;
p = &m;
```

定义常量指针

- 不可通过常量指针修改其指向的内容

```
int n, m;
const int * p = & n;
* p = 5;  //编译出错
n = 4;    //ok
p = &m;
```

定义常量指针

- 不可通过常量指针修改其指向的内容

```
int n, m;
const int * p = & n;
* p = 5;  //编译出错
n = 4;    //ok
p = &m;   //ok, 常量指针的指向可以变化
```

定义常量指针

- 不能把常量指针赋值给非常量指针，反过来可以

```
const int * p1; int * p2;
p1 = p2;     //ok
p2 = p1;     //error
p2 = (int * ) p1; //ok,强制类型转换
```

定义常量指针

- 函数参数为常量指针时，可避免函数内部不小心改变参数指针所指地方的内容

```
void MyPrintf( const char * p ) {
    strcpy( p, "this"); //编译出错
    printf("%s", p);    //ok
}
```

定义常引用

- 不能通过常引用修改其引用的变量

```
int n;
const int & r = n;
r = 5; //error
n = 4; //ok
```

测一测

- 以下说法正确的是：

A. 常引用所引用的变量，其值不能被修改
B. 不能通过常量指针，去修改其指向的变量
C. 常量指针一旦指向某个变量，就不能再指向其他变量
D. 以上都不对

测一测

- 以下说法正确的是：

A. 常引用所引用的变量，其值不能被修改

B. 不能通过常量指针，去修改其指向的变量

C. 常量指针一旦指向某个变量，就不能再指向其他变量

D. 以上都不对

目　录

- 从C到C++
  - 引用的概念和应用
  - const关键字的用法
  - 动态内存分配
  - 函数重载、函数缺省参数
- 类和对象

用 new 运算符实现动态内存分配

- 第一种用法，分配一个变量

```
P = new T;
```

T是任意类型名，P是类型为 T * 的指针

- 动态分配出一片大小为 sizeof(T) 字节的内存空间，并且将该内存空间的起始地址赋值给P

```
int * pn;
pn = new int;
* pn = 5;
```

用 new 运算符实现动态内存分配

- 第二种用法，分配一个数组：

```
P = new T[N];
```

N：要分配的数组元素的个数，可以是整型表达式

- 动态分配出一片大小为 sizeof(T)*N 字节的内存空间，并且将该内存空间的起始地址赋值给P

```
int * pn;
int i = 5;
pn = new int[i * 20];
```

用 new 运算符实现动态内存分配

- 第二种用法，分配一个数组：

```
P = new T[N];
```

N：要分配的数组元素的个数，可以是整型表达式

- 动态分配出一片大小为 sizeof(T)*N 字节的内存空间，并且将该内存空间的起始地址赋值给P

```
int * pn;
int i = 5;
pn = new int[i * 20];
```

```
pn[0] = 20;
pn[100] = 30; //编译没问题。运行时导致数组越界
```

用 delete 运算符释放动态分配的内存

- 用 new 动态分配的内存空间，
- 一定要用 delete 运算符进行释放

```
delete 指针; //该指针必须指向new出来的空间
```

```
int * p = new int;
* p = 5;
delete p;
delete p;   //导致异常，一片空间不能被delete多次
```

用 delete 运算符释放动态分配的内存

- 用 delete 释放动态分配的数组，要加 []

```
delete [] 指针; //该指针必须指向new出来的数组
```

```
int * p = new int[20];
p[0] = 1;
delete [ ] p;
```

测一测

- 表达式 new int 的返回值类型是

A int

B int *

C int &

D void

测一测

- 表达式 new int 的返回值类型是

A int

B int *

C int &

D void

测一测

- 下面小段程序，哪个是正确的：

  A. char * p = new int; p = 'a'; delete p;

  B. int *p = new int[25]; p[10] = 100; delete p;

  C. char * p = new char[10]; p[0] = 'K'; delete [] p;

  D. 都不对

测一测

- 下面小段程序，哪个是正确的：

  A. char * p = new int; p = 'a'; delete p;

  B. int *p = new int[25]; p[10] = 100; delete p;

  C. char * p = new char[10]; p[0] = 'K'; delete [] p;

  D. 都不对

目　录

- 从C到C++
  - 引用的概念和应用
  - const关键字的用法
  - 动态内存分配
  - 函数重载、函数缺省参数
- 类和对象

函数重载

- 一个或多个函数，**名字相同**，然而参数个数或参数类型不相同，这叫做**函数的重载**

函数重载

- 一个或多个函数，**名字相同**，然而参数个数或参数类型不相同，这叫做**函数的重载**

```
int Max(double f1, double f2) { }
int Max(int n1, int n2) { }
int Max(int n1, int n2,int n3) { }
```

- 函数重载使得函数命名变得简单
- 编译器根据**调用语句的中的实参的个数和类型**判断应该调用哪个函数

```
Max(3.4,2.5);//调用(1)
Max(2,4);     //调用 (2)
Max(1,2,3);   //调用 (3)
Max(3,2.4);   //error, 二义性
```

函数的缺省参数

- C++中，定义函数的时候可以让**最右边的连续若干个参数有缺省值**，那么调用函数的时候，若相应位置不写参数，参数就是缺省值

函数的缺省参数

- C++中，定义函数的时候可以让**最右边的连续若干个参数有缺省值**，那么调用函数的时候，若相应位置不写参数，参数就是缺省值

```
void func( int x1, int x2 = 2, int x3 = 3) { }
```

```
int main()
{
    func(10 );      //等效于 func(10,2,3)
    func(10, 8) ;   //等效于 func(10,8,3)
    func(10, , 8); //不行，只能最右边的连续若干个参数缺省
    return 0;
}
```

函数的缺省参数

- 函数参数可缺省的目的在于提高程序的**可扩充性**
- 即如果某个写好的函数要添加新的参数，而原先那些调用该函数的语句，
- 未必需要使用新增的参数，那么为了避免对原先那些函数调用语句的修改，就可以使用缺省参数

函数的缺省参数

- 函数参数可缺省的目的在于提高程序的**可扩充性**
  - 即如果某个写好的函数要添加新的参数，而原先那些调用该函数的语句，
  - 未必需要使用新增的参数，那么为了避免对原先那些函数调用语句的修改，就可以使用缺省参数

```
void func( int x1, int x2 = 2, int x3 = 3) { }
```

```
    func(10 );      //等效于 func(10,2,3)
    func(10, 8) ;   //等效于 func(10,8,3)
    func(10, , 8); //不行，只能最右边的连续若干个参数缺省
    return 0;
```

当然可以！下面是一个C++代码示例，展示了如何使用缺省参数来提高程序的可扩充性。

假设我们有一个函数 `printMessage`，它最初只需要打印一条消息。后来，我们想要扩展这个函数，使其可以指定消息的前缀和后缀，但我们不希望修改之前调用这个函数的代码。

```
#include<iostream>
#include<string>
using namespace std;

//最初的函数，只有一个参数
//void printMessage(const string& message) {
//    cout << message << endl;
//}

//扩展后的函数，增加了前缀和后缀参数，并提供了缺省值
void printMessage(const string& message,
                  const string& prefix = "",
                  const string& suffix = ""){
    cout<< prefix << message << suffix << endl;
}

int main() {
    //最初的调用方式
    printMessage("Hello, World!");
    // 使用新增的前缀和后缀参数
    printMessage("Hello, World!","[Prefix]", "[Suffix]");
    // 仍然可以使用最初的调用方式，不需要修改
    printMessage("This is a test.");
    return 0;
}
```

```
Hello, World!
[Prefix]Hello, World![Suffix]
This is a test.
```

目 录

- 面向对象的基本概念
- 类与对象
- 类成员的访问权限
- 构造函数 & 复制构造函数 & 析构函数
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元

目　录

- 面向对象的基本概念
- 类与对象
- 类成员的访问权限
- 构造函数 & 复制构造函数 & 析构函数
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元

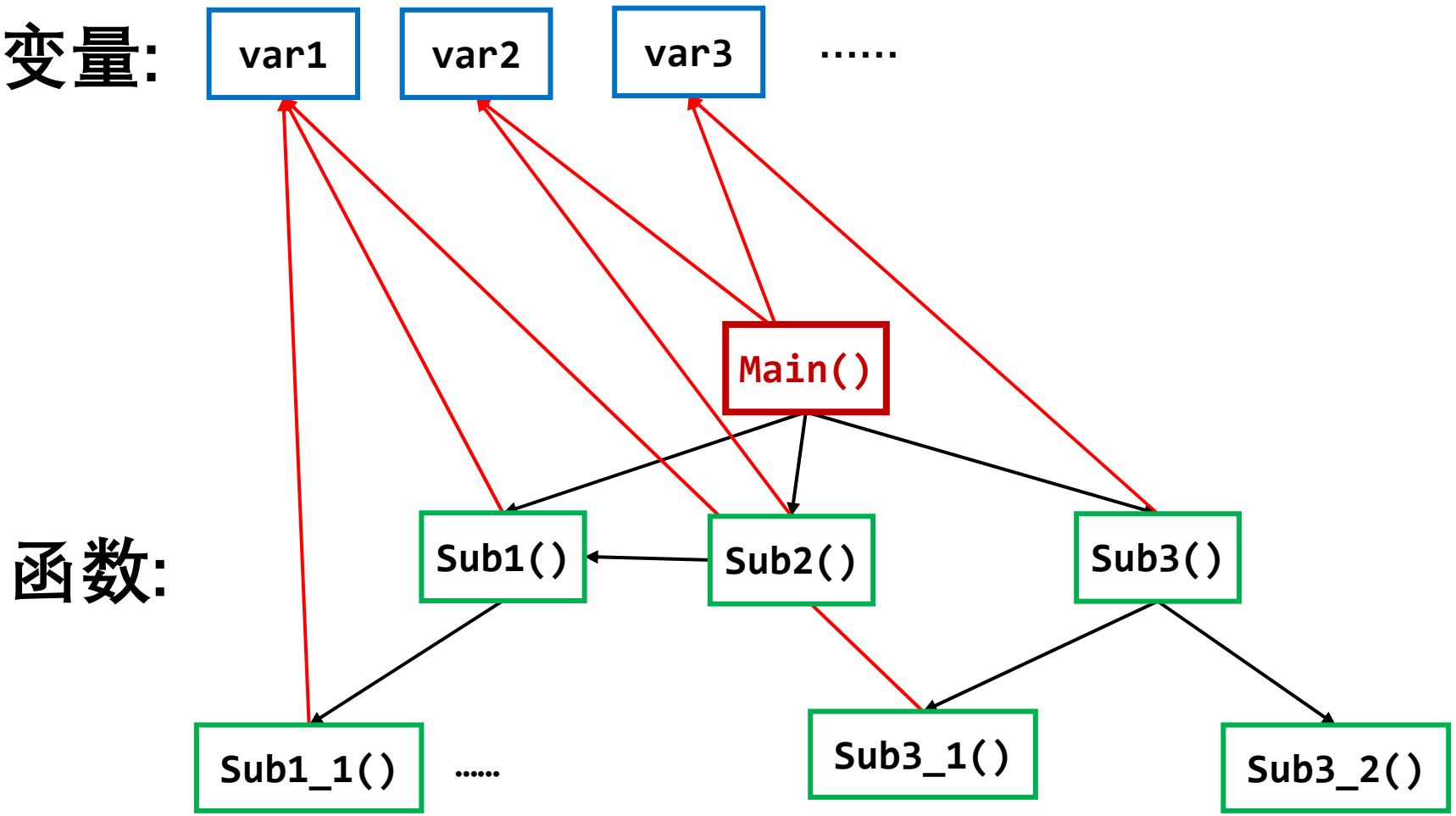
结构化程序设计

- **C语言使用结构化程序设计**

  程序 = 数据结构 + 算法

- 程序由全局变量以及众多相互调用的函数组成
- 算法以函数的形式实现，用于对数据结构进行操作

结构化程序设计



结构化程序设计

- 小明C语言完成了一个“学生成绩管理系统”的课程设计。他设计了一个 struct Student 结构体，用来存放每个同学的信息
- 然后他写了一大堆函数：inputStudent 录入数据，calcAverage 计算平均分，printStudent 打印成绩单，updateScore更新成绩，……一切看起来井井有条。
  - 有一天，教务处新来了一个实习生。他不知道有 updateScore 这个函数，或者他图省事，直接在代码里写了一句：stu1.score = -500; 结果： 数据库崩溃了，学生的成绩变成了负数。
  - 助教老师发来通知：考虑到部分同学成绩偏低，需要把所有不及格（<60分）的同学成绩加上5分，直到及格为止。小明觉得这很简单，于是立刻在 main 函数里写了一个新函数 addFiveToFailed，结果：数据库崩溃了，数组下标越界

结构化程序设计

- 结构化程序设计中，函数和其所操作的数据结构，没有直观的联系
- 随着程序规模的增加，程序逐渐难以理解，很难一下子看出来：
  - 某个数据结构到底有哪些函数可以对它进行操作？
  - 某个函数到底是用来操作哪些数据结构的？
  - 任何两个函数之间存在怎样的调用关系？

结构化程序设计

- 软件业的目标是更快、更正确、更经济地建立软件。
  - 如何更高效地实现函数的复用?
  - 如何更清晰的实现变量和函数的关系?
  - 使得程序更清晰更易于修改和维护。

主要内容

- **面向对象的基本概念**
  - 例子 – 矩形类
  - 三种方式使用
  - 类成员的访问权限

面向对象的程序设计

- **面向对象的程序设计方法：**
  - 将某类客观事物**共同特点**（**属性**）归纳出来

    → 形成一个**数据结构**（用多个变量描述事物的属性）
  - 将这类事物所能进行的**行为**也归纳出来

    → 形成一个个**函数**

    → 这些函数可以用来操作数据结构

C++关键字: 抽象

面向对象的程序设计

- **通过某种语法形式，**

  将**数据结构**和操作该数据结构的**函数**"捆绑"在一起

  → 形成一个"类"

- 使得数据结构和操作该数据结构的算法呈现出显而易见的紧密关系

C++关键字: 抽象

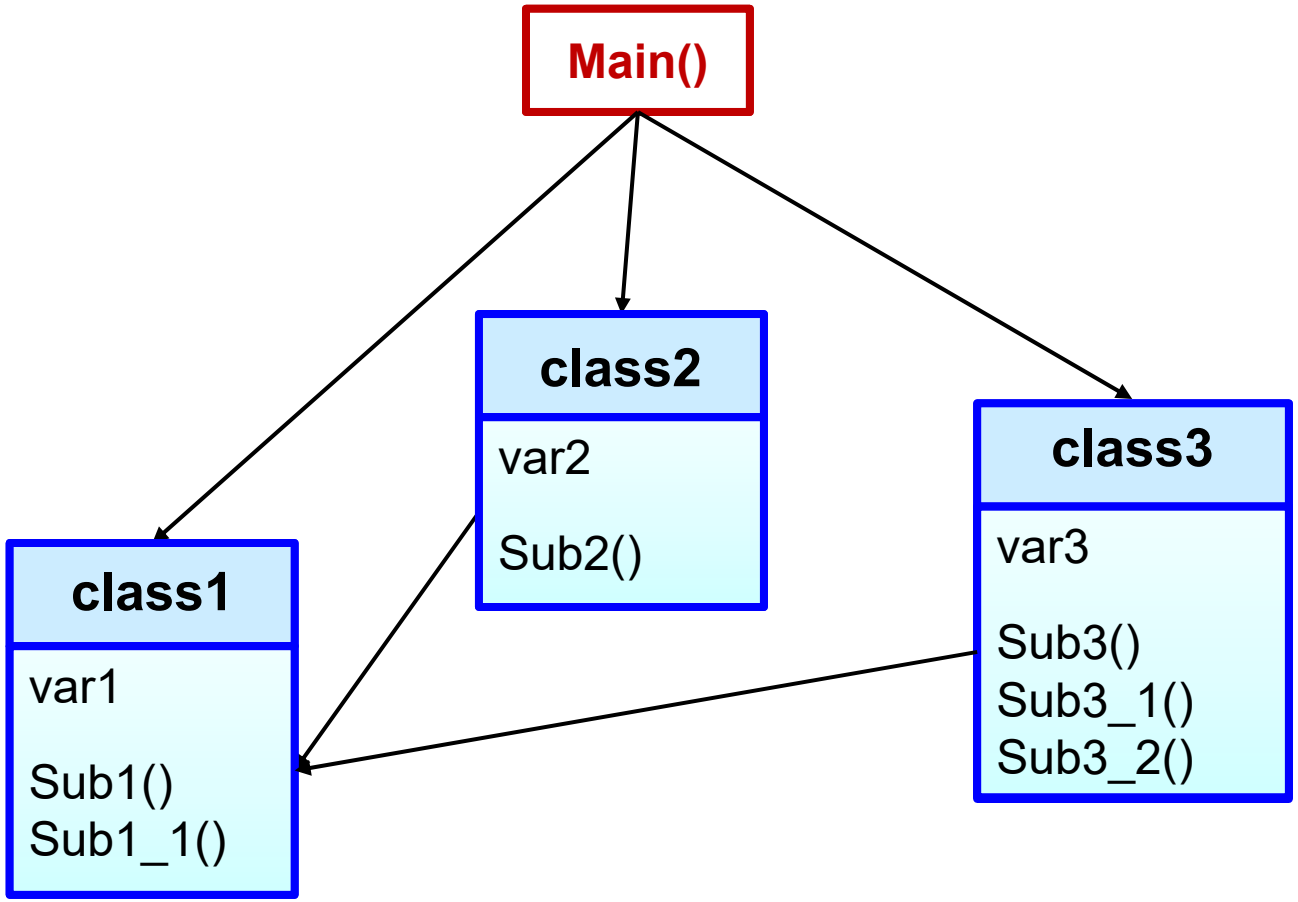
面向对象的程序设计

- 面向对象的程序设计方法，能够较好解决上述问题

  面向对象的程序 = 类 + 类 + … + 类

- 设计程序的过程，就是设计类的过程

面向对象的程序模式


Main()
class2
var2
Sub2()
class3
var3
Sub3()
Sub3_1()
Sub3_2()
class1
var1
Sub1()
Sub1_1()

目　录

- 面向对象的基本概念
- 类与对象
- 类成员的访问权限
- 构造函数 & 复制构造函数 & 析构函数
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元

从客观事物抽象出类

- 写一个程序，输入矩形的长和宽，输出面积和周长
  - 如对于"矩形"这种对象，要用一个类来表示，该如何做"抽象"呢？
  - 矩形的属性就是长和宽
    - 因此需要两个变量，分别代表长和宽
  - 可以对矩形进行哪些操作
    - 矩形可以有设置长和宽，计算面积和计算周长这三种行为
    - 这三种行为，可以各用一个函数来实现，都需要用到长和宽这两个变量

从客观事物抽象出类

- 将长，宽变量和设置长，宽，求面积，以及求周长的三个函数封装在一起，就能形成一个**"矩形类"**
  - 长和宽变量成为该"矩形类"的**"成员变量"**
  - 三个函数成为该类的**"成员函数"**
  - 成员变量和成员函数统称为类的成员
- **类 → 带函数的结构**

从客观事物抽象出类


```
class CRectangle
{
public:
    int w, h;
    int Area() {
        return w * h;
    }
    int Perimeter(){
        return 2 * (w + h);
    }
    void Init(int w_, int h_) {
        w = w_;  h = h_;
    }
}; //必须有分号
```

```
int main( ) {
  int w, h;
  CRectangle  r; //r是CRectangle类的一个对象
  cin >> w >> h;
  r.Init(w, h);
  cout << r.Area() << endl << r.Perimeter();
  return 0;
}
```

对　象

- 通过类，可以定义变量
- 类定义出来的变量，也称为**类的实例**，即**"对象"**
- C++中，类的名字就是用户自定义的类型的名字
- 可以像使用基本类型那样来使用它
  - CRectangle 就是一种**用户自定义**的类型

对象的内存分配

- 和结构变量一样，**对象**所占用的内存空间的大小，

  = 所有成员变量的大小之和
- 对于上面的CRectangle类，**sizeof(CRectangle) = 8**
- 每个对象各有自己的存储空间
- 一个对象的某个成员变量被改变了，不会影响到另一个对象

对象间的运算

- 和结构变量一样，对象之间可以用 "=" 进行赋值
- 不能用 "==" "!=" ">" "<" ">=" "<="进行比较，除非这些运算符经过了 "重载"

使用类的成员变量和成员函数

用法1：对象名.成员名

```
CRectangle r1, r2;
r1.w = 5;
r2.Init(3, 4);
```

- Init函数作用在r2上
- 即Init函数执行期间访问的w和h是属于r2这个对象的
- 执行r2.Init不会影响到r1

使用类的成员变量和成员函数

用法2：指针->成员名

```
CRectangle  r1, r2;
CRectangle * p1 = & r1;
CRectangle * p2 = & r2;
p1->w = 5;
p2->Init(3, 4); //Init作用在p2指向的对象上
```

使用类的成员变量和成员函数

用法3：引用名.成员名

```
CRectangle r2;
CRectangle & rr = r2;
rr.w = 5;
rr.Init(3, 4); //rr的值变了，r2的值也变
```

使用类的成员变量和成员函数

用法3：引用名.成员名

```
    CRectangle r2;
    CRectangle & rr = r2;
    rr.w = 5;
    rr.Init(3, 4); //rr的值变了，r2的值也变

void PrintRectangle(CRectangle & r){
    cout << r.Area() << "," << r.Perimeter();
}
CRectangle r3;
r3.Init(3, 4);
PrintRectangle(r3);
```

```
12,14
```

类的成员函数的另一种写法

- 成员函数体和类的定义分开写

```
class CRectangle
{
    public:
        int w, h;
        int Area(); //成员函数仅在此处声明
        int Perimeter();
        void Init( int w_, int h_ );
};
```

类的成员函数的另一种写法


```
int CRectangle::Area()  {
    return w * h;
}
int CRectangle::Perimeter() {
    return 2 * ( w + h);
}
void CRectangle::Init( int w_, int h_ ){
    w = w_;  h = h_;
}
```

- CRectangle::说明后面的函数是CRectangle类的成员函数，而非普通函数
- 一定要通过对象或对象的指针或对象的引用才能调用

目　录

- 面向对象的基本概念
- 类与对象
- 类成员的访问权限
- 构造函数 & 复制构造函数 & 析构函数
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元

类成员的访问权限

- 结构化程序设计 vs. 面向对象程序设计 → **封装**
- 类成员的**可访问范围**
  - 用**访问范围关键字**来说明类成员可被访问的范围：
    - **private：私有成员，只能在成员函数内访问**
    - **public：公有成员，可以在任何地方访问**
    - **protected：保护成员**

```
class className {
    private:
    //私有属性和函数
    public:
    //公有属性和函数
    protected:
    //保护属性和函数
};
```

类成员的访问权限

类成员的可访问范围

- 如过某个成员前面没有上述关键字，则**缺省地被认为是私有成员**

```
class Man {
    int nAge;//私有成员
    char szName[20];//私有成员
public:
    void SetName(char * szName){
        strcpy(Man::szName, szName);
    }
};
```

以上三种关键字**出现的次数**和**先后次序**都没有限制

类成员的访问权限

- **在类的成员函数内部，能够访问：**
  - 当前对象的全部属性，函数
  - 同类其它对象的全部属性，函数
- **在类的成员函数以外的地方，只能够访问该类对象的公有成员**

类成员的访问权限


```
class CEmployee  {
    private:
        char szName[30]; //名字
    public:
         int salary; //工资
         void setName(char * name);
         void getName(char * name);
         void averageSalary(CEmployee e1, CEmployee e2);
};
void CEmployee::setName(char * name)  {
    strcpy(szName, name); //ok
}
void CEmployee::getName(char * name)  {
    strcpy(name, szName); //ok
}
```

类成员的访问权限


```
void CEmployee::averageSalary(CEmployee e1, CEmployee e2)
{
    salary = (e1.salary + e2.salary)/2;
}

int main()
{
    CEmployee e;
    strcpy(e.szName, "Tom1234567889");//编译错,
                                      //不能访问私有成员
    e.setName("Tom");   // ok
    e.salary = 5000;    // ok
    return 0;
}
```

类成员的访问权限


```
int main()
{
    CEmployee e;
    strcpy(e.szName, "Tom1234567889");//编译错，
                                      //不能访问私有成员
    e.setName("Tom");   // ok
    e.salary = 5000;    // ok
    return 0;
}
```

- 设置私有成员的**目的**
  - 强制对成员变量的访问一定要通过成员函数进行
  - 方便修改成员变量的类型等属性→只需更改成员函数 vs. 否则，所有直接访问成员变量的语句都需要修改
- 设置私有成员的机制叫**隐藏**

类成员的访问权限

- 例如，如将上述程序 → 内存空间紧张的手持设备上
  - 将 szName 改为 char szName[5]，
  - 若 szName 不是私有，那么就要找出所有类似

```
strcpy(man1.szName, "Tom1234567889");
```

  - 这样的语句进行修改，以防止数组越界 → 太麻烦!
- 如果将szName变为私有，
  - 那么程序中就不会出现（除非在类的内部）

```
strcpy(man1.szName, "Tom1234567889");
```

类成员的访问权限

- 所有对 szName 的访问都是通过成员函数来进行，
  - 例如：

```
man1.setName("Tom12345678909887");
```

  - 如需将 szName 改短，也不需将上面的语句找出来修改
  - 只要改 <u>setName**成员函数**</u>，在里面确保不越界即可

用struct定义类

```
struct CEmployee {
    char szName[30];//公有!!
public :
    int salary; //工资
    void setName(char * name);
    void getName(char * name);
    void averageSalary(CEmployee e1, CEmployee e2);
};
```

和用 class 的唯一区别，就是**未说明**是公有还是私有的成员，就是**公有**

本讲小结

- 从C到C++
  - 引用的概念和应用
  - const关键字的用法
  - 动态内存分配
  - 函数重载、函数缺省参数
- 面向对象的基本概念
- 类与对象
- 类成员的访问权限